

IA Aplicada com LLMs

Aula 01 — Redes neurais e a arquitetura Transformer

De onde partimos

```
response = client.chat.completions.create(  
    model="mistral-small-latest",  
    messages=[{"role": "user", "content": "Por que o céu é azul?"}],  
)
```

Dez linhas. Entra pergunta, sai texto coerente.

O que existe do outro lado dessa chamada?

Roteiro

1. **O que é uma rede neural** — a peça elementar e como ela aprende
2. **O que é o Transformer** — a arquitetura de 2017 e o problema que ela resolveu
3. **Qual arquitetura gerou as LLMs** — e por que foi essa

Não vamos treinar um LLM. Vamos saber **o que tem dentro** — o suficiente para depurar, escolher e projetar em cima.

Objetivos

- Explicar a conta de um neurônio e **por que a não-linearidade é indispensável**
- Descrever o treino: perda → gradiente → retropropagação
- Explicar **self-attention** em termos de query, key e value
- Justificar **positional encoding** e **máscara causal**
- Argumentar por que a linhagem **decoder-only** gerou as LLMs

Parte 1

O que é uma rede neural

O neurônio artificial

Não simula uma célula. É uma função de duas linhas:

$$z = w \cdot x + b \qquad a = \sigma(z)$$

- w — **pesos**: quanto cada entrada importa
- b — **viés**: desloca o resultado
- σ — **ativação**: esmaga ou recorta o valor

Uma soma ponderada, seguida de uma função. É isso.

Exemplo: detector de spam

Entradas: nº de links, nº de palavras em CAIXA ALTA, remetente conhecido

$$w = [0,8, 0,6, -2,0] \quad b = -1,5$$

Para $x = [3, 2, 0]$:

$$z = 2,4 + 1,2 + 0 - 1,5 = 2,1 \quad \sigma(2,1) \approx 0,89$$

89% de spam. O peso $-2,0$ derruba a pontuação se o remetente é conhecido.

Ninguém escreveu esses números — eles vêm do treino.

Sem ativação, tudo colapsa

Duas camadas lineares:

$$y = W_2(W_1x + b_1) + b_2 = \underbrace{(W_2W_1)}_{W'}x + \underbrace{(W_2b_1 + b_2)}_{b'}$$

$$y = W'x + b'$$

Uma única camada. Cem camadas, mil camadas: sem ativação, tudo vira uma reta.

A não-linearidade é o que permite empilhar.

O caso que provou isso: XOR

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Desenhe **uma reta** separando os 1 dos 0.

Não existe — os 1 estão em cantos opostos. **Não é linearmente separável.**

Minsky & Papert, 1969 → a área congelou por uma década.

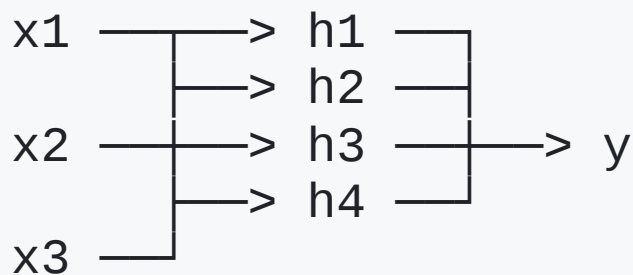
Ativações que importam

Função	Onde aparece
Sigmoid	saída de classificadores binários
Tanh	RNNs clássicas
ReLU — $\max(0, z)$	padrão do deep learning; FFN do Transformer original
GELU	GPT-2 em diante
SwiGLU	Llama, Mistral, LLMs atuais

Reter: **a ativação existe para introduzir não-linearidade**. A escolha específica é otimização empírica.

Da peça à rede

Vários neurônios lado a lado = **camada**. Camadas empilhadas = rede profunda.



Junte os pesos numa matriz e a camada inteira é:

$$a = \sigma(xW + b)$$

Um produto de matrizes por camada. É por isso que GPU virou o hardware da IA.

Contando parâmetros

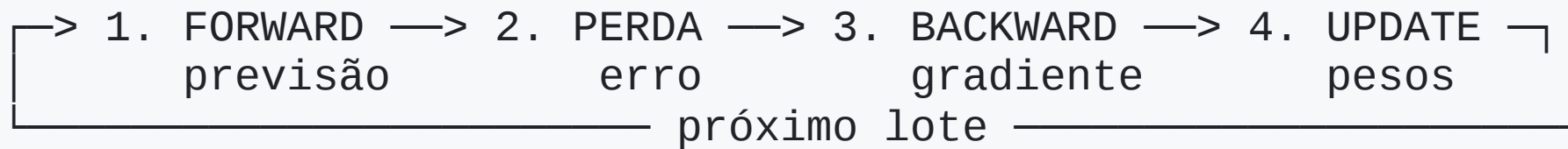
Camada de n entradas e m neurônios: $n \times m$ pesos $+$ m vieses.

Rede $784 \rightarrow 128 \rightarrow 10$ (dígitos manuscritos):

Camada	Parâmetros
$784 \times 128 + 128$	100.480
$128 \times 10 + 10$	1.290
Total	101.770

"7 bilhões de parâmetros" é esta mesma contagem — setenta mil vezes maior.

Como uma rede aprende



Repetido milhões de vezes.

Treinar um LLM de 7B é **este mesmo loop**.

A perda: quão surpreso o modelo ficou

Entropia cruzada — a perda usada para treinar LLMs:

$$L = - \sum y_{\text{real}} \log(y_{\text{previsto}})$$

Probabilidade dada à palavra correta	Perda
0,01	$-\log(0,01) \approx 4,6$
0,90	$-\log(0,90) \approx 0,1$

Pune com força a **confiança errada**.

Treinar um LLM = minimizar isso sobre trilhões de tokens.

Gradiente descendente

A perda é uma paisagem: cada eixo é um peso, a altitude é o erro.

Você está no escuro e só sente **a inclinação sob os pés**.

$$w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w}$$

- **gradiente** — a direção de subida mais íngreme (vamos ao contrário)
- η — **taxa de aprendizado**: grande demais salta o vale, pequena demais demora

Na prática: **lotes** (*batches*) de algumas centenas de exemplos.

Retropropagação

O peso da 1ª camada afeta a perda por um caminho longo. Regra da cadeia:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1}$$

O erro nasce na saída e é **distribuído para trás**; cada peso recebe sua parcela de culpa.

Consequência: o gradiente é um *produto* de muitos fatores.

Fatores $< 1 \rightarrow$ encolhe até zero (**vanishing gradient**). Fatores $> 1 \rightarrow$ explode.

Metade dos truques do deep learning existe para controlar isso.

XOR resolvido: 2 camadas, numpy puro

```
W1 = rng.normal(0, 1, (2, 4)); b1 = np.zeros((1, 4))
W2 = rng.normal(0, 1, (4, 1)); b2 = np.zeros((1, 1))

for epoca in range(10001):
    a1 = sigmoid(X @ W1 + b1)                # 1. forward
    a2 = sigmoid(a1 @ W2 + b2)
    d2 = (a2 - y) / len(X)                  # 3. backward
    d1 = (d2 @ W2.T) * a1 * (1 - a1)
    W2 -= taxa * (a1.T @ d2); W1 -= taxa * (X.T @ d1) # 4. update
```

época	0	perda	0.7568	previsões:	[0.001	0.999	0.999	0.002]
época	10000	perda	0.0011	rótulos:	[0	1	1	0]

Quatro neurônios ocultos e uma não-linearidade bastaram.

Parte 2

O Transformer

Texto quebra a rede comum

MLP tem entrada de tamanho fixo. Texto não:

- "Oi" tem 2 letras; um contrato tem 40 mil palavras
- A **ordem** importa
- As **dependências são longas**:

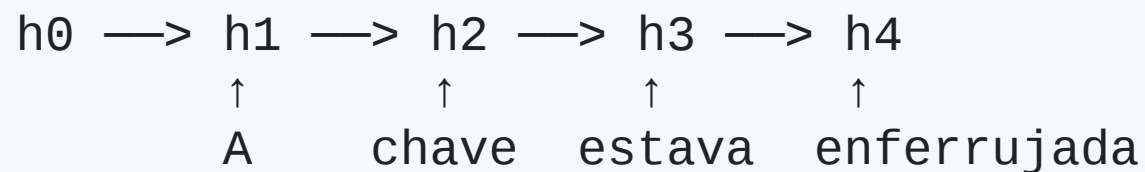
"A **chave** que o técnico deixou sobre a mesa da recepção do prédio antigo, depois de três horas de manutenção, **estava** enferrujada."

17 palavras entre "chave" e "estava".

RNN: memória por estado oculto

Processa um token por vez, carregando um estado h que resume o passado:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$$



Aceita qualquer tamanho. LSTM (1997) melhorou com portas de esquecer/guardar.

Dominou tradução automática por anos.

Os três problemas da RNN

1. **Memória que desbota** — tudo passa por um vetor de tamanho fixo; o começo se perde
2. **Gradiente que desaparece** — retropropagar por 500 passos = produto de 500 fatores
3. **Sequencialidade** — para calcular h_{100} preciso de $h_{99} \dots$

Os dois primeiros são sobre **qualidade**.

O terceiro é sobre **escala**: mil tokens = mil passos em série. A GPU fica ociosa.

Enquanto o treino é sequencial, não dá para jogar mais GPU no problema.

2017: *Attention Is All You Need*

Vaswani et al., Google, NeurIPS 2017. O título é a tese: **jogue a recorrência fora.**

O problema real era prático:

- O Google traduzia bilhões de frases/dia
- Treinar LSTM levava **semanas** — não paralelizava
- O modelo original: 65M parâmetros, **12 horas em 8 GPUs**, estado da arte em EN → DE

O ganho: sem recorrência, todos os tokens são processados **ao mesmo tempo** no treino.

Acidente histórico: foi inventado para **tradução**. A generalidade foi descoberta depois.

Self-attention: uma busca aprendida

Cada token olha para todos os outros e remonta sua própria representação.

Vetor	Papel
Query (q)	"o que eu estou procurando"
Key (k)	"o que eu tenho a oferecer"
Value (v)	"a informação que eu entrego se for escolhido"

1. Compare minha *query* com cada *key* → um score por token
2. Softmax nos scores → pesos que somam 1
3. Some os *values* ponderados por esses pesos

Por que isso resolve linguagem

"O gato não subiu na árvore porque **ele** estava com medo"

Ao processar "ele":

- a *query* de "ele" bate **forte** com a *key* de "gato"
- bate **fraco** com a de "árvore"
- a nova representação de "ele" incorpora o *value* de "gato"

Correferência resolvida **sem nenhuma regra gramatical programada** — só pesos aprendidos.

A fórmula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V$$

Peça	O que faz
$QK^{\top}$	cada query $\times$ cada key $\rightarrow$ matriz $n \times n$ de "quem olha quem"
$\sqrt{d_k}$	normaliza a escala dos scores
softmax	cada linha vira distribuição de probabilidade
$\cdot V$	média ponderada dos values

Nenhum loop. Nenhuma dependência do passo anterior. Tudo paralelizável.

Por que dividir por $\sqrt{d_k}$

Com d_k grande, os produtos escalares crescem em magnitude.

Scores grandes no softmax $\rightarrow$ distribuição quase **one-hot** (toda a massa num token só).

Nessa região o gradiente é $\sim$ zero $\rightarrow$ **o treino trava**.

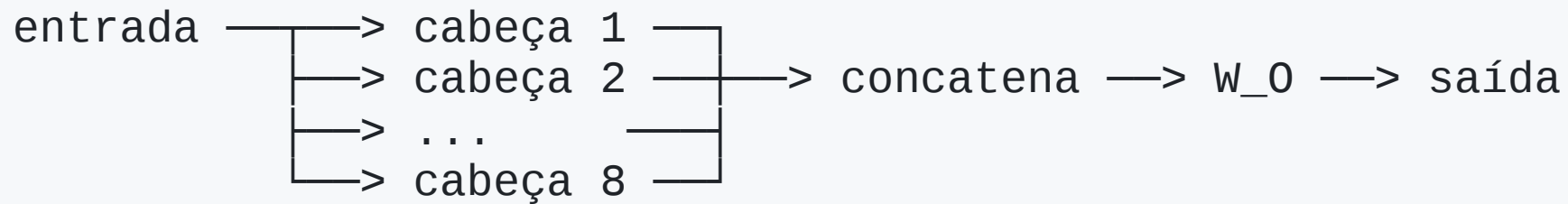
Dividir pela raiz da dimensão mantém os scores numa faixa saudável.

Daí o nome: **scaled** dot-product attention.

Múltiplas cabeças

Linguagem tem várias relações simultâneas: concordância, correferência, negação...

Uma atenção só teria que escolher uma.



Cada cabeça tem seu próprio W_Q , W_K , W_V numa fatia menor da dimensão.

Em modelos treinados, algumas cabeças **se especializam** de forma interpretável.

A atenção é cega à ordem

Permute os tokens de entrada: $Q' = PQ$, $K' = PK$, $V' = PV$.

A saída sai **igual, apenas permutada**. A atenção trata a entrada como **conjunto**.

"o homem morde o cão" $\equiv$ "o cão morde o homem"

Solução: positional encoding — injetar a posição no vetor de cada token.

- Original: senos e cossenos de frequências diferentes, somados ao embedding
- Atuais (Llama, Mistral): **RoPE** — *rotaciona* query e key pela posição

É aqui que mora o trabalho por trás de "agora com 128 mil tokens de contexto".

O resto do bloco

FFN — MLP de 2 camadas aplicado **em cada posição separadamente**

- a atenção mistura *entre* tokens; a FFN processa *cada* token
- concentra $\sim 2/3$ **dos parâmetros** do modelo

Conexão residual — $x + \text{Atenção}(x)$

- caminho curto para o gradiente; permite empilhar dezenas de camadas

Normalização (LayerNorm / RMSNorm)

- estabiliza o treino

Máscara causal

Um LLM prevê o **próximo** token. Se pudesse ver o futuro, leria a resposta.

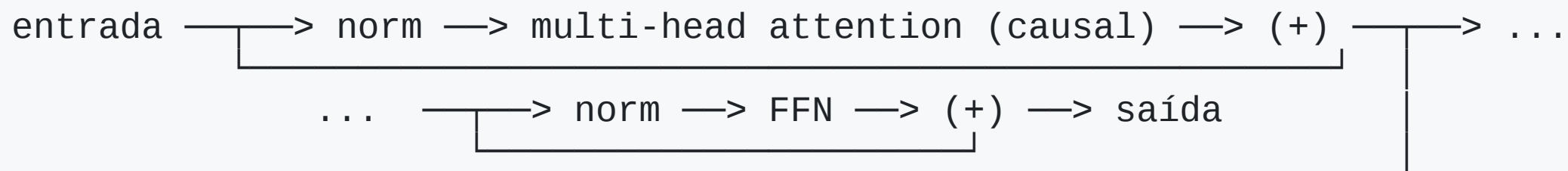
Antes do softmax: scores do futuro recebem $-\infty \rightarrow$ peso zero.

	0	gato	subiu	nele
0	[1.00	0.00	0.00	0.00]
gato	[0.40	0.60	0.00	0.00]
subiu	[0.03	0.33	0.64	0.00]
nele	[0.09	0.32	0.58	0.01]

Triangular inferior. Cada linha soma 1.

É esta máscara que separa um **decoder** de um **encoder**.

O bloco completo



E o modelo inteiro:

```
texto -> tokenização -> embeddings + posição -> [bloco] × N ->  
-> norm final -> projeção no vocabulário -> P(próximo token)
```

A saída não é uma palavra — é uma distribuição sobre todo o vocabulário.

O preço: custo quadrático

Cada token olha todos os outros → matriz $n \times n$.

Tokens no contexto	Entradas na matriz
1.000	1 milhão
10.000	100 milhões
100.000	10 bilhões

Dobrar o contexto quadruplica o custo.

Daí vem: limite de janela, prompt longo caro, *context engineering*, e **RAG** vencendo "joga o documento todo no contexto".

O Mistral mitiga com *sliding window* e *grouped-query attention*.

O trade-off fundamental

	RNN	Transformer
Memória	barata (vetor fixo)	quadrática no contexto
Paralelismo no treino	impossível	total

O Transformer trocou **sequencialidade** por **custo quadrático**.

Ótimo negócio para o hardware disponível — e o limite que você negocia todo dia.

Parte 3

A arquitetura que gerou as LLMs

Três linhagens do Transformer

O original (2017) era **encoder-decoder**, para tradução. A comunidade separou:

	Encoder-only	Decoder-only	Encoder-decoder
Exemplo	BERT	GPT, Llama, Mistral	T5, BART
Atenção	bidirecional	causal	mista
Treino	preencher lacunas	prever próximo token	texto → texto
Gera texto livre?	não	sim	sim
Uso hoje	embeddings, busca	assistentes, agentes	conversão específica

Encoder-only não é inferior

BERT vê o texto inteiro de uma vez.

→ Entende melhor do que escreve.

Você vai reencontrá-lo no curso: os modelos de **embedding** usados em busca semântica e **RAG** são tipicamente encoder-only.

Por que decoder-only venceu

Não foi elegância. Foi uma propriedade do **objetivo de treino**.

1. O rótulo já está no dado

"Prever o próximo token" não precisa de anotação humana → treinar na internet inteira

2. Uma tarefa que contém todas as outras

Prever bem a próxima palavra exige gramática, fatos, tradução, raciocínio

Resumir e classificar viram **casos particulares de continuar texto**

3. Uma interface única

Texto entra, texto sai → nasce o **in-context learning**

Descrever a tarefa no prompt, sem retreinar nada

Descoberto, não projetado. É a base deste curso inteiro.

Escala: quando quantidade virou qualidade

Leis de escala (Kaplan et al., 2020) — a perda cai de forma **previsível** com parâmetros, dados e computação.

Previsível o suficiente para estimar o desempenho **antes** de treinar.

Chinchilla (Hoffmann et al., 2022) — os modelos da época eram grandes demais para os dados usados.

Regra prática: **~20 tokens de treino por parâmetro**

Por isso um 7B bem treinado compete com modelos muito maiores da geração anterior.

A conta de um LLM

Por bloco decoder-only: atenção $4d^2$ (W_Q, W_K, W_V, W_O) + FFN $8d^2$

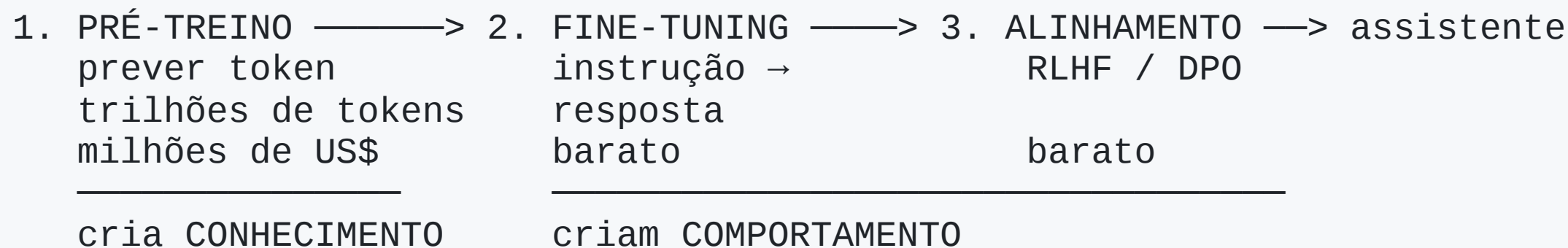
$$\text{parâmetros} \approx 12 \cdot N \cdot d_{model}^2$$

Modelo	N	d_{model}	Estimado
Transformer original	6	512	0,02 B
GPT-2 small	12	768	0,08 B
GPT-3	96	12288	173,95 B
Mistral 7B	32	4096	6,44 B

Acerta o GPT-3 ("175B"). Erra o Mistral (7,24 B real) — SwiGLU e GQA fogem da fórmula.

De preditor de tokens a assistente

Um modelo só pré-treinado **não é** um assistente: ele imita texto da internet.



Marco: InstructGPT (2022) → ChatGPT, nov/2022.

Nada de novo em arquitetura — o mesmo decoder-only, agora alinhado.

Linha do tempo

Ano	Marco
1958	Perceptron
1986	Retropropagação
1997	LSTM
2014	Seq2seq + atenção (como acessório)
2017	Transformer — fim da recorrência
2018	BERT e GPT-1 — a bifurcação
2020	GPT-3 + leis de escala → in-context learning
2022	InstructGPT → ChatGPT
2023—	Llama, Mistral, Claude, GPT-4

Voltando ao começo: o que roda na API

```
client.chat.completions.create(model="mistral-small-latest", messages=[...])
```

1. Mensagens formatadas e **tokenizadas**
2. Cada token → vetor (**embedding**) + posição
3. Dezenas de **blocos decoder-only**: attention causal + FFN
4. Projeção no vocabulário → **distribuição de probabilidade**
5. Um token é **sorteado**
6. Token anexado à entrada → **volta ao passo 3**
7. Pára no token de fim ou no limite

Duas consequências que você usa todo dia

1. A resposta varia entre execuções

O passo 5 é um **sorteio**. `temperature` controla o quanto ele arrisca.

2. O modelo não "consulta" nada

Sem banco, sem arquivo, sem internet. Só os **pesos congelados** no treino e o **texto do contexto**.

→ É exatamente essa limitação que motiva **RAG** e os **agentes com ferramentas**.

Síntese — rede neural e Transformer

- **Rede neural** = somas ponderadas + não-linearidade
Sem a ativação, tudo colapsa numa reta
- **Treinar** = forward → perda → retropropagação → update
Um LLM usa esse mesmo loop
- **RNN** travou na **sequencialidade** — o que impedia usar a GPU e, portanto, escalar
- **Transformer** jogou a recorrência fora e ficou só com a **atenção**:
$$\text{softmax}(QK^{\top} / \sqrt{d_k})V$$

Síntese — das peças às LLMs

- Peças: multi-head, positional encoding, FFN ($\frac{2}{3}$ dos parâmetros), residual, máscara causal
- Custo **quadrático** no contexto → limite de janela, RAG, *context engineering*
- **Decoder-only** venceu porque o rótulo já está no dado, a tarefa contém as outras e a interface é única → **in-context learning**
- A arquitetura é de **2017**. Mudaram escala, dados e **alinhamento**

Para ir além

Papers

- Vaswani et al. — *Attention Is All You Need* (2017) — seções 3.1–3.3
- Hoffmann et al. — *Training Compute-Optimal LLMs* (2022) — Chinchilla
- Jiang et al. — *Mistral 7B* (2023)

Didático

- Karpathy — *Let's build GPT: from scratch, in code, spelled out* (YouTube)
- Alammr — *The Illustrated Transformer*
- 3Blue1Brown — série *Neural Networks*

Nesta aula: `notas-de-aula/01-redes-neurais-e-transformer.md` —
desenvolvimento completo, exemplos executáveis e 4 exercícios resolvidos.